

CI/CD with Jenkins

Lab 1

1	LAB SETUP	1
2	BASIC JENKINS FREESTYLE PROJECT WITH SHELL COMMANDS	2
3	ADDING A FILTER TO VIEW THE JOBS IN THE DASHBOARD	6
4	BASIC JENKINS FREESTYLE PROJECT WITH WINDOWS BATCH COMMANDS.....	7
5	CONNECTING TO THE MAIN SERVER VIA SSH	9
6	INTRODUCING MAVEN FOR BUILDING JAVA PROJECTS	10
7	INSTALL AND CONFIGURE MAVEN PLUGIN	14
8	CREATING A GITHUB REPO FOR THE PROJECT	16
9	CHECKING / INSTALLING GIT / GITHUB PLUGINS FOR JENKINS	20
10	BASIC JENKINS JOB TO INTEGRATE WITH GITHUB	21
11	CONFIGURING PERIODIC BUILDS AND SCM POLLING	29
12	CHAINING JOBS / PROJECTS	32
13	CONFIGURING WEBHOOKS WITH GITHUB.....	34
14	SETTING UP A BITBUCKET REPO	38
15	CONFIGURING WEBHOOKS WITH BITBUCKET.....	42
15.1	EXERCISE: BITBUCKET WEBHOOK CONFIGURATION	43
16	CONFIGURING EMAIL NOTIFICATION	44

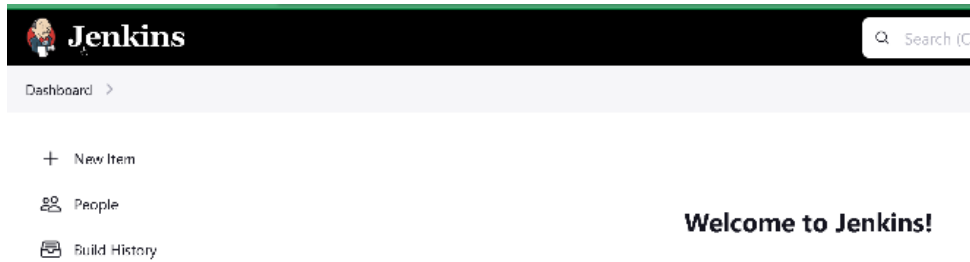
1 Lab setup

You should have been given a lab guide detailing the various configuration details for AWS EC2 instances that you will be using for the duration of this workshop.

Verify that you can access the main Jenkins UI at your designated server:

<http://server-ip-address:port-number>

Login with the provided username / passwd combination at Jenkins UI main page. After successful login, the main UI should look something like this:



In all the labs that follow, *userx* refers to your assigned username in the lab guide.

2 Basic Jenkins freestyle project with Shell commands

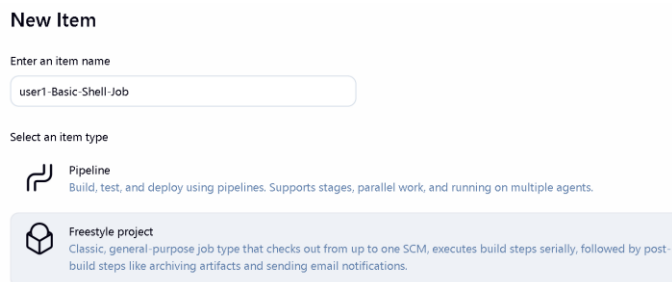
A Jenkins job (sometimes called project) is a user-configured description of work which Jenkins should perform, which is usually building a piece of software. Each job consists of various stages, which in turn contain a group of steps, where a step is a single fundamental task that tells Jenkins what to do. Typical examples of these tasks include gathering dependencies, compiling, archiving, or transforming code, and testing and deploying code in different environments.

Jenkins supports several types of jobs: such as freestyle projects, pipelines, multi-configuration projects, folders, multibranch pipelines, and organization folders. The two most commonly used jobs which we will cover extensively in this workshop are freestyle projects and pipelines. The other kinds of jobs are specializations of pipelines used for very specific use case scenarios.

From the main Jenkins dashboard, select + New Item or Create a Job.

create a new Item and name it: *userx-Basic-Shell-Job*

Make this a Freestyle Project and select Ok.



The configuration for this job is divided into several different stages / sections, which are also found in other projects. Each stage incorporates specific actions / steps to be performed. This step functionality is provided from the plugins that already installed on the main Jenkins controller, and any additional specialized functionality required will be obtained by adding more appropriate plugins.

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment**
- Build Steps
- Post-build Actions

Type some random info in the Description for this project in the General section.

Configure

General

 General

 Source Code Management

Description

Using basic Linux shell commands in a simple job

In the Build Steps section, choose Add Build step -> Execute Shell commands, and add in the following Linux shell commands:

Build Steps

 Execute shell ?

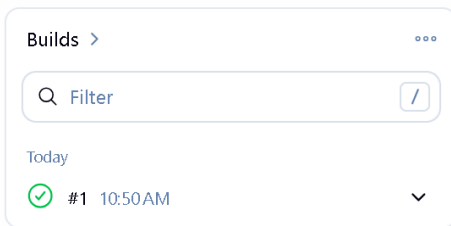
Command

See [the list of available environment variables](#)

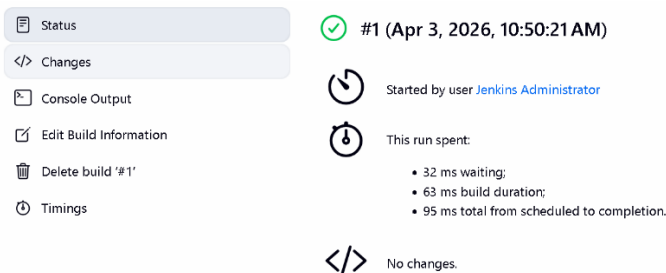
```
echo "Current active account is $(whoami)"
echo 'Creating a new directory if it does not exist and a new file'
mkdir -p awesome
cd awesome
NAME=Richard
echo "Hello, $NAME. The date is $(date)" > newfile
ls -l
pwd
echo "The content of newfile is "
cat newfile
```

Click Save.

Back at the main Job dashboard, click Build Now to run a build of the job. This should succeed and you should see a green circle next to a number (the particular build attempt) indicating success in the Build History.



Click on the Green Circle to get more detailed info on that particular build attempt.



The most important option you can explore from the navigation pane on the left is the Console Output, which is the log of all the activities that occurred during the build.

Some points to note:

- The name of the logged in user (`Jenkins Main Administrator`) is shown
- The rest of the listing is the console output corresponding to the execution of the various shell commands. You can see that the active user account that is running the Jenkins main controller service is called `jenkins`, which is system user account (this is different from normal user accounts).
- The complete path of the workspace folder is shown. The default installation directory for Jenkins on a Linux machine is `/var/lib/jenkins`. Within this installation directory is a folder called `workspace`, and within `workspace` are all the workspace folders corresponding to all the jobs / items that you create in Jenkins. In this case, the complete path to the current workspace would be will be: `/var/lib/jenkins/workspace/user1-Basic-Shell-Job`. The final folder name will be the same as the job name, and all items / files related to that job will be placed within this workspace folder.
- All build-related activity for that job (creation of new directories, files, etc) will take place relative to the workspace folder for that job by default, unless the full path for another location is explicitly specified in one of the steps / tasks of the job.
- If you have any commands that manipulate directories or files, these directories or files will be relative to the workspace folder of that job. So, for e.g. if the `newfile` file that was created by one of the shell commands is within the `awesome` directory, which itself is within the top-level workspace folder.
- If you wish to access locations or files outside of the workspace folder, then remember to specify an absolute path: e.g. `/home/user1`
- However, remember that the Jenkins service is running as the `jenkins` user, so if you execute a shell command that attempt to access a directory / file / process that this user does not have permissions to access, an access error will occurs just as in the case for a standard Linux command.

Click on Edit Build Information to provide more random information on this particular build attempt, and then click Save.

Status

Changes

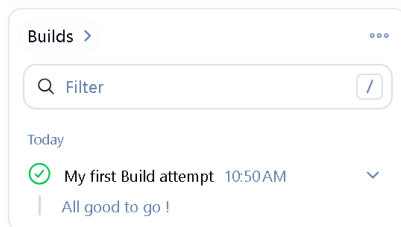
Console Output

Edit Build Information

DisplayName ?
 My first build attempt

Description ?
 All good to go !

Return back to the main page for the job. Notice that the display name and description for this build attempt is now shown in the Build History.

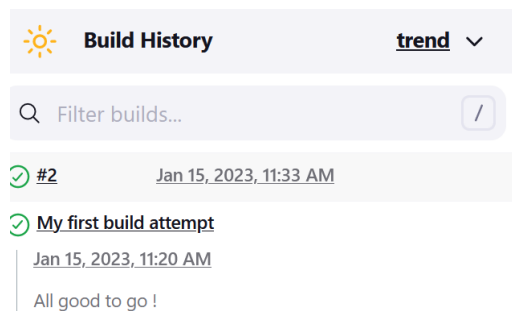


If you click on Workspace, you will be able to get a UI that allows you to navigate the workspace folder structure (which was created as part of the build process) as well as download the entire folder to our local machines in a zip format. This would be useful here since the Jenkins service will typically be running on a remote server in a real life scenario. Go ahead and try that here.

Workspace of user1-Basic-Shell-Job on Built-In Node



Click Build Now again to perform another build attempt. This should succeed and the build attempt number is shown in the Build History.



Repeat a build a few more times to see the build attempt list grow in the Build History. The Permalinks section in the middle provide links for you to navigate to different build attempts depending on their status (stable, successful, completed).

✓ **userx-Basic-Shell-Job**

using basic

Permalinks

- [Last build \(#4\), 2.7 sec ago](#)
- [Last stable build \(#4\), 2.7 sec ago](#)
- [Last successful build \(#4\), 2.7 sec ago](#)
- [Last completed build \(#4\), 2.7 sec ago](#)

More info on [build status concepts](#) and terms:

So far for our very simple job, all the builds fall into this category.

If you now return to the main Dashboard, you will be able to see all the jobs that have been created on the Jenkins server by all logged in users up to this point of time (this means all the other participants in your assigned group, since they are all also working on the same Jenkins server).

All +

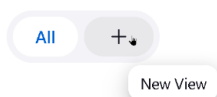
S	W	Name ↓	Last Success
✓	☀	user1-Basic-Shell-Job	13 min #4
✓	☀	user2-Basic-Shell-Job	9.1 sec #1

The various icons next to the job listing demonstrate the status of that job, and you can get more detailed info by just hovering your mouse over the particular icon.

As the lab continues to progress the list of jobs available for inspection will become very long and it may be cumbersome to identify the jobs that you created. We can impose a filter on the list of jobs in the dashboard to simplify this.

3 Adding a filter to view the jobs in the dashboard

Select New View.



Give it the name: `view-for-userx`
And select the Type as List View.

New view

Name

view-for-user1

Type

☒ List View

Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

Select Create.

In the Jobs section, select Use a regular expression to include jobs into the view
And specify the regular expression: `userx-.*`

☒ Use a regular expression to include jobs into the view ?

Regular expression

`user1-.*`

Add Job Filter ▾

Click Ok.

You should now only be able to see jobs that start with `userx-` in the main Dashboard.

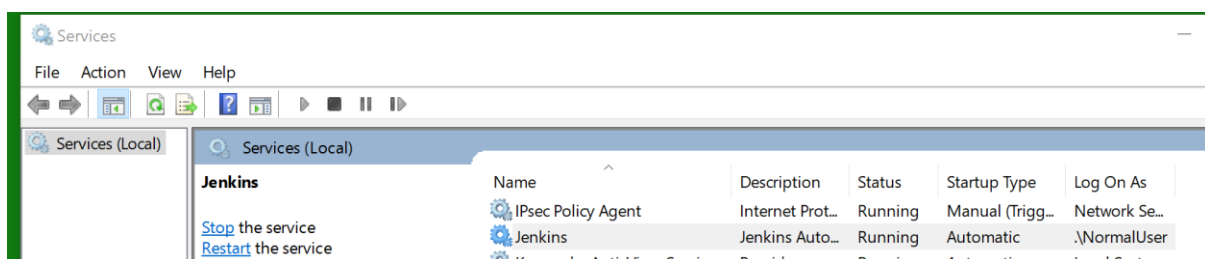
All view-for-user1 +

S	W	Name ↓
✓	☀	user1-Basic-Shell-Job

You can always click on All to return back to view all available jobs in the main Dashboard.

4 Basic Jenkins freestyle project with Windows batch commands

This is an optional exercise if you have managed to install Jenkins on your Windows machine successfully. Typically, it is installed on Windows as a service and will start on the default port of 8080. You can access it via the Services app.



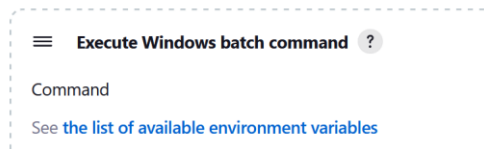
If Jenkins service is already running, you can open a browser tab to

<http://localhost:8080>

where you will see the main login page for the Jenkins UI, and you can login using the admin username / password combination that you registered during the installation process.

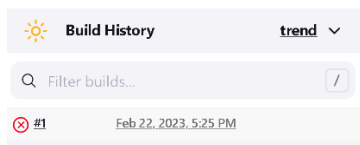
Repeat the process of creating a job that we did earlier, but this time in the Build Steps section, choose Add Build step -> Execute Windows Batch command

Build Steps



Now, see what happens if you attempt to reuse the list of shell commands that we entered previously in the lab on Linux shell command in the area for Windows batch command. Save and run a build on this job.

This time you will get an error for the build



And if you check the Console Output for this particular build, you will notice that errors are due to certain Linux shell commands not being recognized in Windows (ls, pwd, cat)

Now Configure this job again with proper DOS commands as shown below:

```
echo "Hi there from Jenkins"
echo "I love Jenkins ! Its the most awesome CI tool ever ... " >
hello.txt
echo "Showing contents of current job folder"
dir
type hello.txt
```

Save and build again. This time the build succeeds, because we are using DOS commands which can be interpreted and executed correctly by the Windows machine that the Jenkins server is running on. You can the Console Output to verify the actions that occur.

The [complete list of DOS command line](#) commands:

The list of [most frequently used](#) command line commands:

Some [basic tutorials](#) to working with the DOS command prompt:

Now if you repeat the process of taking these DOS commands and attempting to use them in the Execute Shell command section of the job that we created earlier on the Jenkins server running on a

Linux machine, we will get similar errors for the same reason (DOS commands are not recognized and cannot be interpreted by the Linux Bash shell).

This demonstrates that there are certain aspects of a job configuration that are platform specific, although most aspects are platform independent. Later on, we will see when we are working with pipelines on how to execute generic file / directory operations in a general way through a Jenkins job regardless of the platform that Jenkins is running on.

5 Connecting to the main server via SSH

We will establish a Bash shell to an existing user account (`mainuser`) on the Main Server so that we can work with some sample Java projects that have already been placed there.

First open Windows Powershell (`powershell`) or Powershell 7 (`pwsh`)

To connect to Bash Shell in Main Server / Secondary Server, execute in either shell:

```
ssh mainuser@server-IP
```

Type Yes to any prompts related to `continue connecting (yes/no/[fingerprint])?`

Enter the password in the lab guide given to you earlier.

When you are logged in you should see a Bash prompt similar to:

```
mainuser@main-server
```

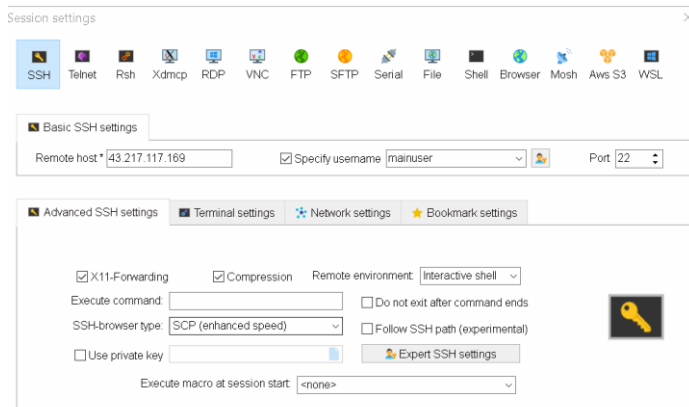
You will type all your Linux shell commands into this prompt. Type:

```
ls -l
```

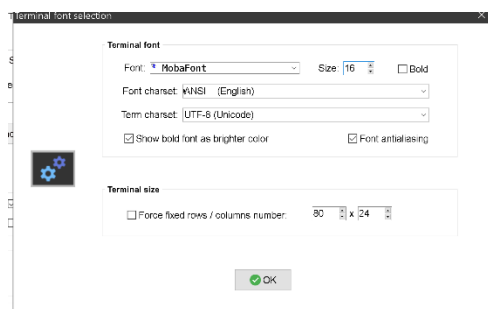
You should be able to see 15 subdirectories (all under the ownership of the current user `mainuser`) with names corresponding to your assigned usernames.

You will establish a connection to the Main Server using the MobaXTerm client so that you can easily navigate and view the contents of the folders corresponding to your assigned usernames, and also transfer files between your local machine and the Main Server.

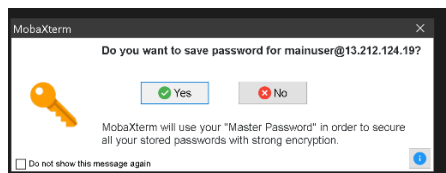
Configure a new SSH connection with the IP address of the Main Server, the `mainuser` account and port 22 (default port for SSH). Also set browser type to SCP (enhanced speed).



You can also set terminal font settings to get a larger font (for e.g. 16) for the shell font size when you successfully login:



Click Ok to connect and type the password when prompted. Select Yes to the prompt to save the password.



With the connection established, you will have a navigation pane on your left. You can navigate through the contents of the folder of your assigned username - similar to the case of Windows Explorer.

Navigate with care and be careful not to accidentally delete any of the files and directories of other user's folders.

6 Introducing Maven for building Java projects

Different programming languages / frameworks utilize different build tools. The most popular build tool for most modern day Java projects is [Maven](#), which is an evolution from an earlier build tool, Ant.

In the folders corresponding to your usernames, there are two folders: `bitbucketprojects` and `githubprojects`. Inside `githubprojects` there are two more folders, which are the root folders of a Maven Java project.

We will work with Maven using the first project: `simple-java-app`.

In the Bash shell, navigate into this folder by typing:

```
cd ~/userx/githubprojects/simple-java-app
```

First, you can check the version of Maven installed on this machine with:

```
mvn -version
```

When working from the command line, you invoke Maven directly using the command line tool `mvn` and passing the required arguments.

Each of the 3 core build lifecycles in Maven (`default`, `clean` and `site`) is defined by a different [list of build phases \(or stages\) in the lifecycle](#).

A particular build phase is performed by executing one or more tasks. Each of these tasks is handled by a plugin goal. A plugin goal may therefore be bound to zero or more build phases. If a goal is bound to one or more build phases, that goal will be called in all those phases.

The bindings for the `clean` and `site` lifecycle phases are fixed. The bindings for the default life cycle phases depending on the `packaging` value.

We can invoke Maven by specifying a phase in any of the build life cycles such as `compile` or `package`

The first thing we will typically do in a standard project is to clean up the artifacts (classes, JARs, etc) from a previous build operation. To do that we can use the `clean` phase from the `clean` life cycle. Type:

```
mvn clean
```

If this is the first time you are running Maven, a whole bunch of related plugins and dependencies will be downloaded from the central Maven repository and stored in the local cache. This may take some time to complete, depending on the capacity of your broadband connection. Once they are available in the local cache, Maven will reference them there for future executions of this phase, thus removing the need for costly network access.

We can also specify two or more phases to be executed: they will be executed in the order they appear.

```
mvn clean compile
```

This performs a clean and also a compilation of the source code. The bytecode classes are now placed in a specific directory for this project (by default for most Java projects this will be `\target\classes`). You can check for the bytecode classes there through MobaXTerm client.

Most projects will ship with some test code to test the main application classes. At the most basic, these will take the form of unit tests implemented using some unit testing library (JUnit for the case of Java projects) - notice that this was included in the dependency list of the POM.

To test the compiled source code, we can type:

```
mvn test
```

Console output from the Maven build indicates that the tests were successful, as we are just implementing some very basic dummy tests here.

Finally we will produce a build artifact that we can directly execute. For the case of a Java project, this is typically a JAR file or a WAR file (Web Archive for deploying a web app to a server such as Tomcat).

To accomplish this, we can type:

```
mvn package
```

Notice that the tests are run as part of the `package` phase, since the `test` phase precedes the `package` phase. This makes sense since we want to ensure that the code base passes all tests before it is deployed to a production environment: a key requirement in the CI part of a CI/CD pipeline in DevOps.

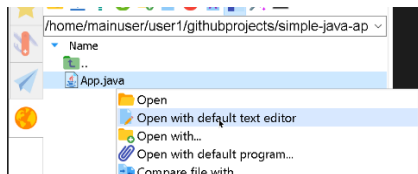
We can also see the directory where the JAR artifact is generated in from the Maven console output. We can execute the artifact directly with:

```
java -jar target/my-app-1.0-SNAPSHOT.jar
```

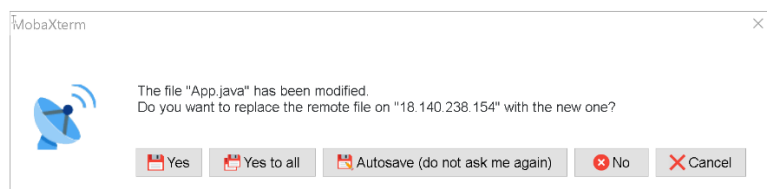
This single line output is from the source code of the main class for this simple Java project at:

```
src/main/java/com/mycompany/app/App.java
```

You can use MobaXTerm's default editor to open it for editing:



Let's make a minor modification to the source code. Change the parameter of the `System.out.println` to some other random value, for e.g. "Jenkins is awesome". Save the changes (Ctrl+S) and select Yes to the prompt that appears.



You can verify that the changes have actually been made by directly viewing the content of this file from the CLI:

```
cat src/main/java/com/mycompany/app/App.java
```

In the shell, we can again repeat the `clean` (to remove all files from the previous build) and then generate the build artifact again with:

```
mvn clean package
```

Now we can execute the newly generated build artifact again with:

```
java -jar target/my-app-1.0-SNAPSHOT.jar
```

Verify that your changes have taken effect. Repeat this a few more times (but don't change the value of the String MESSAGE yet).

Finally, lets simulate an error occurring in the unit tests. Back in the same file:

```
src/main/java/com/mycompany/app/App.java
```

Change the value of String MESSAGE to some other value, for e.g. "Dummy String"
Save and exit.

Now if you check the corresponding unit test for the previous class at:

```
src/test/java/com/mycompany/app/AppTest.java
```

You will notice that the change you made will cause the statement
`assertEquals("I love Jenkins", app.getMessage());`
to fail.

In the command prompt, perform a clean (to remove all files from the previous build) and then generate the build artifact with:

```
mvn clean package
```

This time the Maven output is in red, indicating the tests have failed. Also, notice that there is no JAR file generated in the `target` directory, since the build process did not complete successfully.

If you check the content of `target/surefire-reports`, you will see two files in different formats (*.txt and *.xml) which contain information pertaining to the tests that failed

Return back to the app source code file:

```
src/main/java/com/mycompany/app/App.java
```

Restore the value of String MESSAGE to its original value: "I love Jenkins"
Save and exit.

Now repeat the Maven goals again:

```
mvn clean package
```

This time the build process completes without error, and we obtain the newly generated build artifact and can execute it again with :

```
java -jar target/my-app-1.0-SNAPSHOT.jar
```

The other main alternative to Maven for Java build tools is [Gradle](#)

There are tools that perform some or part of Maven build functionality (e.g. dependency and package management) for other programming languages / frameworks:

[Nuget](#) is the package manager for C# [.NET Core projects](#)

[MSBuild](#) is the build tool for [.NET C# applications](#)

[MSTest](#) is one of the testing frameworks for [.NET C# applications](#)

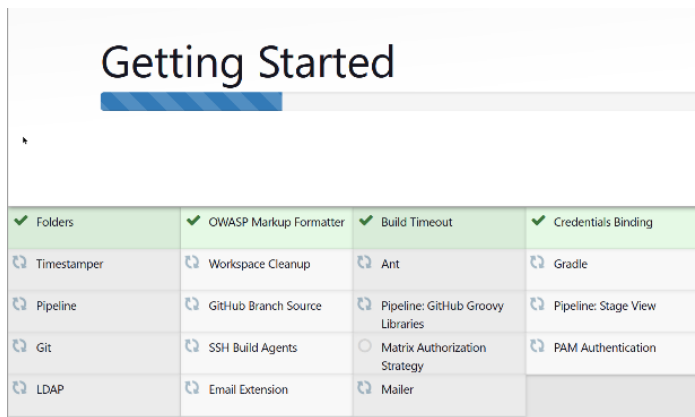
[CMake](#) is popular build tool for [C++ projects](#)

[Pip](#) is the most commonly used tool for [installing packages and setting up virtual environments](#) in Python.

7 Install and configure Maven plugin

Plugins are key to the functionality of Jenkins. When a plugin is installed, it provides functionality to Jenkins by adding available steps that can be used in the actions / options to be selected in the key stages of a standard free style project (General, SCM, Build Triggers, Build Environment, Build Steps, Post Build Actions). These steps / options represent a fundamental unit of action that is performed by Jenkins.

There are a set of suggested plugins which are typically installed during initial installation of Jenkins,



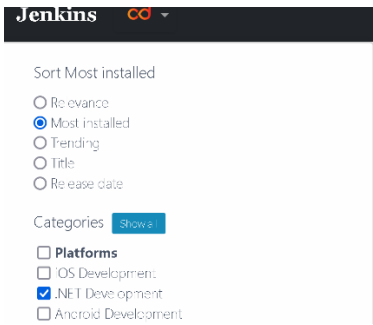
The screenshot shows the 'Getting Started' page of Jenkins. At the top, there is a title 'Getting Started' with a blue progress bar below it. Below the progress bar is a table of suggested plugins. The table has four columns, each with a green checkmark icon and a title. The first column is 'Folders', the second is 'OWASP Markup Formatter', the third is 'Build Timeout', and the fourth is 'Credentials Binding'. Each column contains a list of plugins with a blue icon to the left of the plugin name.

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding
 Timestampper	 Workspace Cleanup	 Ant	 Gradle
 Pipeline	 GitHub Branch Source	 Pipeline: GitHub Groovy Libraries	 Pipeline: Stage View
 Git	 SSH Build Agents	 Matrix Authorization Strategy	 PAM Authentication
 LDAP	 Email Extension	 Mailer	

In addition there is a large ecosystem of plugins accessible from the Update Center which can then be further installed to add additional functionality

<https://plugins.jenkins.io/>

You can perform a filter on the plugins based on various categories from the main search page:

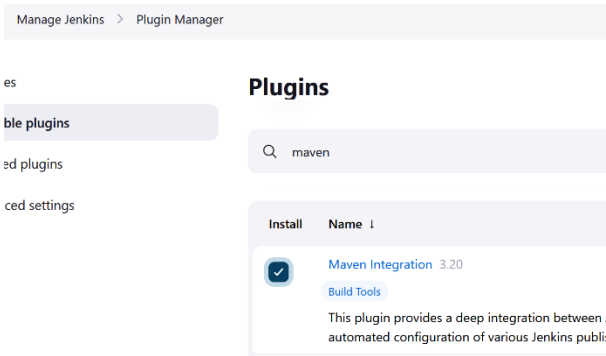
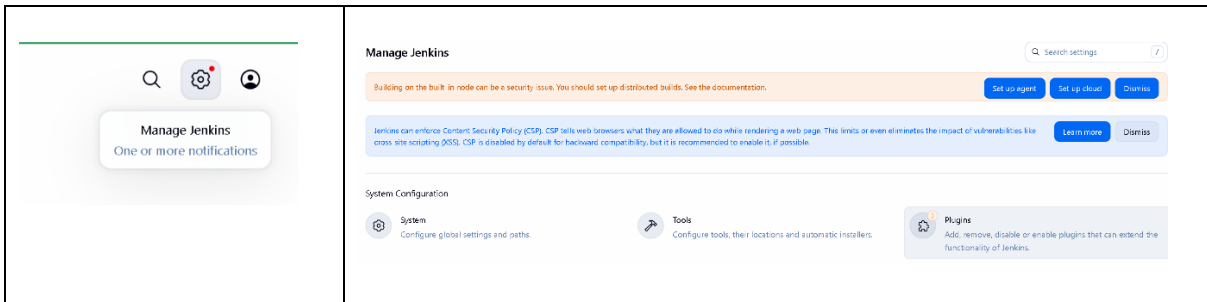


Notice that you have plugins to implement the functionality for the various build tools related to .NET C# application development: MSBuild, MSTest, Nuget, etc

The same goes for Python: do a search for this term.

The action below of installing a plugin only needs to be performed once by a user with Admin Privilege.

From the main Jenkins Dashboard, select Manage Jenkins from the upper right hand corner. Then select -> Plugins -> Available Plugins, search for Maven Integration and select the item.



Select Install and click Restart Jenkins when installation is complete and no jobs are running while the download is proceeding. You may need to refresh the webpage to verify that the installation is complete.

--	--

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

Javadoc ✓ Success

Maven Integration ✓ Success

Loading plugin extensions ✓ Success



Jenkins is getting ready to work

Your browser will reload automatically when Jenkins is ready.

The restart may take some time to complete or may occasionally fail. In Linux, you can start / stop the Jenkins service from the CLI via:

```
sudo systemctl start / stop jenkins
```

You can check the status of the Jenkins service using the command:

```
sudo systemctl status jenkins
```

You may have to log back in again once the restart is complete.

From the main Jenkins Dashboard, Manage Jenkins -> Plugins -> Installed Plugins, and verify that Maven Integration plugin is installed.

Name ↓	Enabled
Maven Integration plugin 3.25 This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit. Report an issue with this plugin	☒

From the main Jenkins Dashboard, Manage Jenkins -> Tools (in the System Configuration section), click Add Maven and give it a suitable name: `jenkins-maven` and select version `3.9.14` and enable Save Automatically, then click Apply and Save.

+ Add Maven

Maven

Name

jenkins-mavens

☒ Install automatically ?

Install from Apache

Version

3.9.14

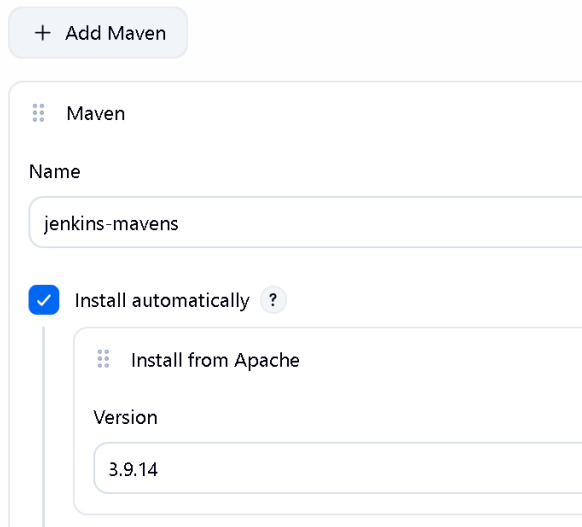
8 Creating a GitHub repo for the project

Login to your GitHub account.

Create a new repository and give it the same name as the Java project that we have been working with so far: `simple-java-app`

Leave it as a public repository (default setting)

DO NOT initialize the repository with anything. Simply click Create repository.



The screenshot shows a Jenkins configuration page for adding a Maven plugin. At the top is a button labeled '+ Add Maven'. Below it is a section titled 'Maven' with a grid icon. Under this section, there is a 'Name' field containing 'jenkins-mavens'. Below the name field is a checkbox labeled 'Install automatically' which is checked, followed by a question mark icon. Below this is another section titled 'Install from Apache' with a grid icon. Under this section, there is a 'Version' field containing '3.9.14'.

Back in the Bash shell, make sure you are in the folder `userx/githubprojects/simple-java-app`

If not, you can change to it with:

```
cd ~/userx/githubprojects/simple-java-app
```

First, we remove any previously existing generated artifacts

```
mvn clean
```

Git is already installed on the server. First, we check that this folder is not a proper Git repo yet with

```
git status
```

An error should indicate that this project folder is not yet a Git repo. If it is, you can simply delete the `.git` folder in the project folder.

Next initialize it as a Git repo with all the files in the project folder with the following sequence of command in the Git Bash shell (you can type them in one at a time):

```
git init
git add .
git status
```

Before creating the first commit in this newly initialized repository, you need provide an identify for the individual creating the commit (the commit author). Since we are going to be pushing our local repo to populate the GitHub repo, we need to specify a correct GitHub identity (name and email) so that the commit contents are pushed from the local repo have their authorship / origin correctly identified. Go to

<https://github.com/settings/profile>

Public profile

Name

Victor Tan

Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.

Public email

victor.tan.33@gmail.com

✕ Remove

You can manage verified email addresses in your [email settings](#).

and note down the name and email address there. Use them in the configuration commands below:

```
git config --local user.name "your name"
```

```
git config --local user.email "your email address"
```

IMPORTANT:

We are configuring these details at the local level so that they apply only to this repository, rather than at the user account level since in this workshop we have multiple users are using the same user account `mainuser`, but with different GitHub username / passwords.

Double check that these configuration variable values have been set properly at the local level, which will be in the file `.git\config` in the current working directory

```
git config --list
```

Then we can create our first commit.

```
git commit -m "Added all project files in first commit"
```

```
git status
```

Return to the main page of your new GitHub repo that you just created. Copy the HTTPS URL shown there:

Quick setup — if you've done this kind of thing before

 Set up in Desktop

 or

HTTPS

SSH

<https://github.com/victor-tan-hk/simple-java-app.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#),

We will refer to this URL as `remote-url` in the commands to follow.

Back in the Bash shell in `simple-java-app`, and type:

```
git remote add origin remote-url
```

Check that the origin handle is set to point to the correct repo URL with:

```
git remote --verbose
```

We can also optionally configure auto color scheme for the Git Bash shell

```
git config --global color.ui auto
```

Before you can push the contents of the newly initialize repo to your remote GitHub repo, you must first obtain a [classic personal access token \(PAT\)](#):

For Select scopes, select the following 2 checkboxes:

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☐ repo:status	Access commit status
☐ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo:invite	Access repository invitations
☐ security_events	Read and write security events

☒ admin:repo_hook	Full control of repository hooks
☐ write:repo_hook	Write repository hooks
☐ read:repo_hook	Read repository hooks

When done, click Generate Token.

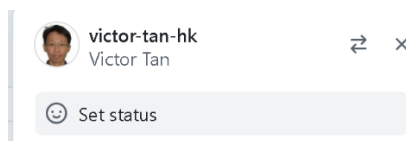
Once the token generation page appears, make sure you copy down the PAT to NotePad++ or somewhere where you can access it later before you navigate away from the page.

Finally, push the entire contents of the local repo (including all its branches) to the remote repo with:

```
git push -u origin --all
```

You will be prompted for your username and password:

Note: Your username (in this case, `victor-tan-hk`) may be different from your actual name (Victor Tan) associated with the GitHub account.



Username for 'https://github.com': *yourusername*

Password for 'https://githubaccountname@github.com': *PAT*

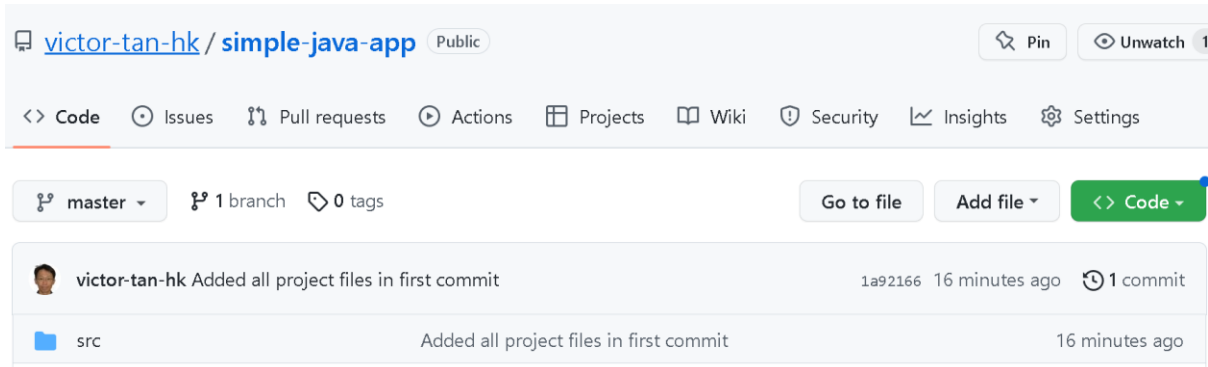
Copy and paste in the PAT from where you saved it earlier. Be very careful to do this slowly and correctly.

Assuming you have entered the correct GitHub user account name and PAT, the command should succeed with appropriate success messages in the Git Bash shell.

```
Enumerating objects: 23, done.
```

```
Counting objects: 100% (23/23), done.  
...  
...  
branch 'master' set up to track 'origin/master'.
```

Refresh the main repo page, you should see the contents that you have just pushed up.



Back in the Git Bash shell, type the following commands to verify that the remote tracking branches have been set up and that the local and upstream master are both in sync with each other.

```
git remote show origin
```

```
git status
```

Other users (for e.g. team members of a project team) can now clone this remote repo to a local repo on their local machines and work on it via a branching workflow.

9 Checking / installing Git / GitHub plugins for Jenkins

There are a variety of Git related plugins for Jenkins, which should have already been installed during the installation of Jenkins itself if you had selected the Install Suggested Plugins in the installation process itself.

To check, go to the main Jenkins Dashboard, Manage Jenkins -> Plugins -> Installed Plugins and search for the term: `git`

These are the plugins that should be installed which we will be using in subsequent labs. If they do not show up in your list, then go to Available Plugins and install them in the same way we have done previously.

Q git	
Name ↓	Enabled
Git 4.14.3 This plugin integrates Git with Jenkins. Report an issue with this plugin	☒
Git client 3.13.1 Utility plugin for Git support in Jenkins Report an issue with this plugin	☒
GitHub 1.36.0 This plugin integrates GitHub to Jenkins. Report an issue with this plugin	☒
GitHub API Plugin 1.303-400.v35c2d8258028 This plugin provides GitHub API for other plugins. Report an issue with this plugin	☒
GitHub Branch Source Plugin 1696.v3a_7603564d04 Multibranch projects and organization folders from GitHub. Maintained by CloudBees, Inc. Report an issue with this plugin	☒
Pipeline: GitHub Groovy Libraries 38.v445716ea_edda_ Allows Pipeline Groovy libraries to be loaded on the fly from GitHub. Report an issue with this plugin	☒

You can initially configure Git in Manage Jenkins -> Tools, to ensure that the correct version of Git is used by Jenkins.

Git

Git installations

Git

Name

Default

Path to Git executable ?

git

☐ Install automatically ?

Add Git ▾

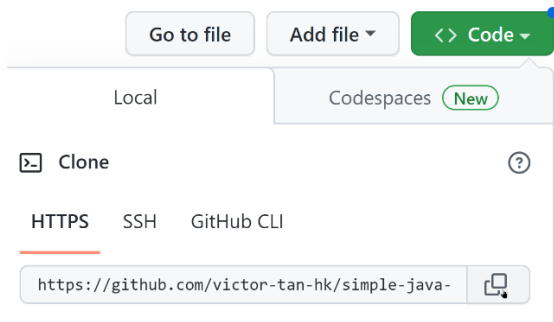
Typically, you can just use the default installation of Git (whether Linux / Windows), which should already be immediately accessible for standard installations of GIT (via the PATH environment variable).

10 Basic Jenkins job to integrate with GitHub

From the main Jenkins dashboard, create a new Item and name it: `userx-Basic-GitHubJob`. Make this a Freestyle Project and select Ok.

In the Configure section, go to Source Code Management and select Git

Return to the main page of your GitHub repo, click on the Green Code button and copy the HTTPs URL:



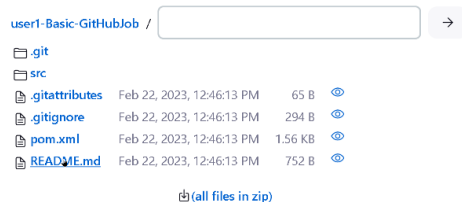
Paste the URL into the Repository URL on the Jenkins configuration page.



Click on Save and perform a build.

The Console Output for this build attempt will show that the remote repo contents have been cloned into the workspace folder for this current job (`workspace\userx-Basic-GitHubJob`). You can navigate to the contents of this folder using the Workspace item in the navigation pane of the main area for the job to verify this.

Workspace of user1-Basic-GitHubJob on Built-In Node



In the main area for the job, select Configure, go to Build Steps and add a Build Step -> Invoke Top Level Maven Targets:

Select the Maven Version that you configured previously and add the goals: `clean package`

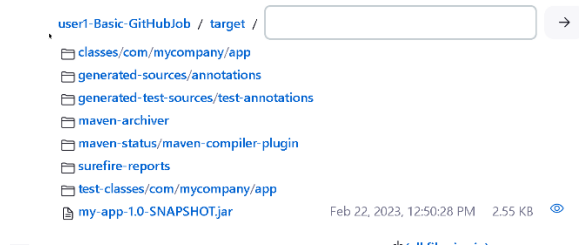


Click Save, then perform a Build.

The build may take some time to complete because the new installation of Maven that you completed earlier will be downloaded the dependencies it needs. When the build completes successfully, the Console output should show the log related to all these dependencies being downloaded, with the JAR artifact being generated successfully at the end.

You can check in the Workspace item from the navigation pane to verify the final JAR artifact was generated successfully.

Workspace of user1-Basic-GitHubJob on Built-In Node



Returning back to the main area for the job, select Configure, go to Build Steps and add another Build Step -> Execute Shell. Type this:

```
echo "*****"
echo "Executing generated JAR"
echo "*****"

java -jar target/my-app-1.0-SNAPSHOT.jar
```

Click Save and run a build again. Verify from the Console Output that the JAR was executed successfully with its corresponding output.

The locally installed JDK is directly accessible in the Shell build step (`java -jar ...`) without any further configuration for this purpose since the path to the Java binary is already on the `$PATH` environment variable for the machine that the Jenkins server is running on.

Nevertheless, you can also explicitly configure its location in the same way as you did for Maven: Manage Jenkins -> Tools

To find the value for `JAVA_HOME` environment variable (which specifies the installation directory for JDK on the current machine), type into the shell:

```
echo $JAVA_HOME
```



When are you done just click Save.

We will now make a change in the local project folder, create a commit from it, push this commit to the remote master and run the build again on our Jenkin server.

Return to the project folder `simple-java-app`

Let's make a minor modification to the source code of the main class for this simple Java project at:

```
src/main/java/com/mycompany/app/App.java
```

Change the parameter of the `System.out.println` to some other random value, for e.g. "Modified from local project !"

Save and exit.

In the Bash shell in this project folder, check that the modifications have being made:

```
git status
```

Add the modifications to a new commit with an appropriate message:

```
git commit -am "My first change in the local project folder"
```

And if we check the status again

```
git status
```

We can see that the latest commit on the master branch is ahead of the remote master by one commit, as we expect.

Before we push the latest commit to the master on the remote GitHub repo, we can optionally configure temporary caching of the username and PAT at a local repository level for the same user account `mainuser` that everyone is working with. This eliminates the need to constantly have to reauthenticate for every push operation. The commands for this are:

```
git config credential.useHttpPath true
```

```
git config credential.helper "store --file .git/.credentials"
```

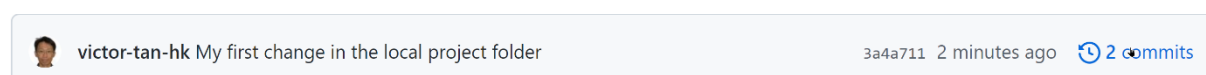
We can then push this latest commit to the master on the remote GitHub repo with:

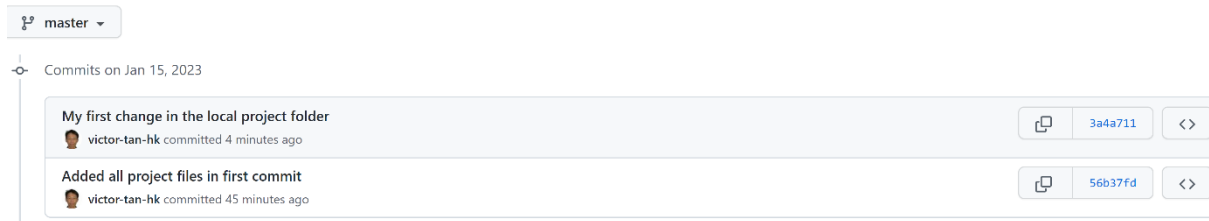
```
git push
```

Enter your GitHub account username and PAT again to authenticate. If everything is correct, you should messages indicating that the changes have been pushed successfully to the GitHub repo.

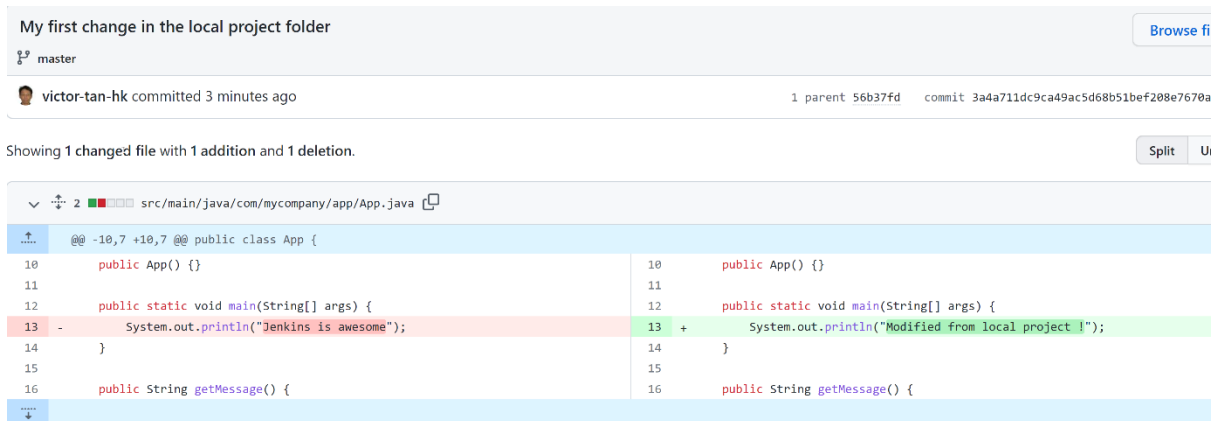
The username and PAT should also be appropriately cached so that the next push will proceed immediately without further prompting at the CLI.

Back on the GitHub repo main page, click on the Commits link to zoom in to the main commits page





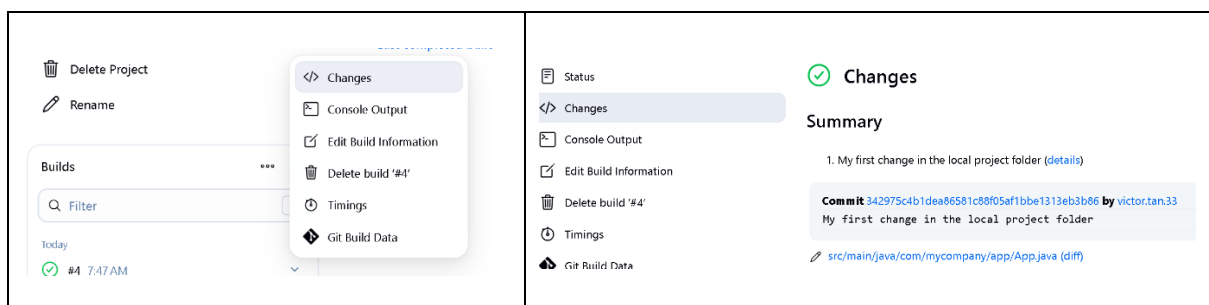
Then click on the hash of the particular commit to see its details.



We can see the latest commit and how it has changed from the previous (initial commit)

Returning back to Jenkins, perform another build of the same job. The latest changes to the remote repo will be pulled down to the local workspace folder, and a build will be run to generate a new build artifact and it will be executed, with the output reflecting the latest change correctly.

You should also be able to view the latest changes corresponding to this latest build which is due to the most recently pushed commit to the repository.



Continue to make some more random changes to the parameter of the `System.out.println` in the source code of the main class for this simple Java project at:

```
src/main/java/com/mycompany/app/App.java
```

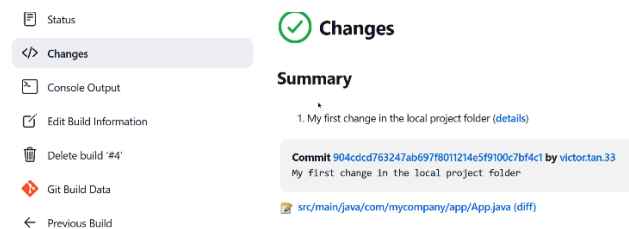
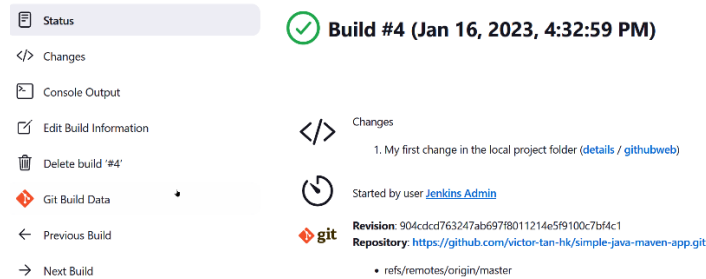
Then commit the changes and push the new commit to the remote repo:

```
git commit -am "My XXXXX change in the local project folder"
```

```
git push
```

Then run a build again on the same job in Jenkins and verify from the Console Output that the JAR execution shows that the artifact was built from the latest changes pulled down from the remote GitHub repo.

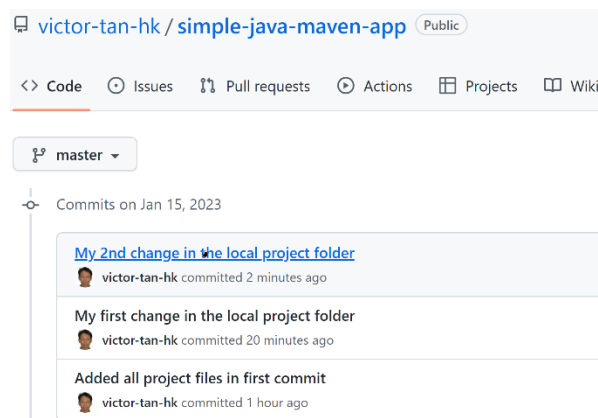
You should be able to see the changes to the remote repo this via the Changes item in the left hand menu and also in the main Status item.



At any point in time, you can check the commit history of the master branch in the local project folder with:

```
git log --oneline
```

And this history is also visible from the main commits page on the GitHub repo.



In the main area for the job, select Configure, go to Post-Build Actions, and add an action -> Publish JUnit test result report

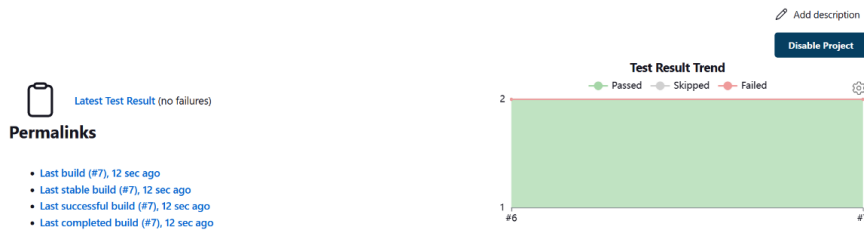
Add in this for the Test Report XMLs fileset field: `target/surefire-reports/*.xml`

This is the location for where the test report XMLs are located relative to the workspace folder

Click Save and perform a Build again.

Click on Status on the main area for the job, or else simply refresh the page. A test result trend should now appear, keeping track of the test results. If you do not see a completely green area, refresh the page or rerun the build again.

Project BasicGitHubDemo



In the main page for the latest build, you should see a link Tests which leads to page with details on the Test Results up to that point in time (e.g. how many pass, how many fail, the difference, etc)

Jenkins / users BasicGitHubJob / #6

Status ✓ #6 (Apr 4, 2026, 9:31:44 AM)

Changes

Console Output

Edit Build Information

Delete build '#6'

Timings

Git Build Data

Tests

Previous Build

Started by user Jenkins Administrator

This run spent:

- 3 ms waiting;
- 10 sec build duration;
- 10 sec total from scheduled to completion.

Revision: c4965da1c04c274a055004927c1e46ff4b7bfa

Repository: <https://github.com/victor-tan-hk/simple-java-app.git>

- refs/remotes/origin/master

Tests (no failures)

No changes.

Tests 2 ✓ 2 Run 0.13 sec View Refresh

All tests are passing!
Nice one! All 2 tests are passing.

All Tests

Package	Failed	Skipped	Passed	Total	Duration
com.mycompany.app	0	0	2	2	71 ms

Return back to the project at: `simple-java-app`

Let's simulate an error occurring in the unit tests. In the file:

`src/main/java/com/mycompany/app/App.java`

Change the value of String MESSAGE to some other value, for e.g. "Dummy String"
Save and exit.

As usual commit the changes and push the new commit to the remote repo:

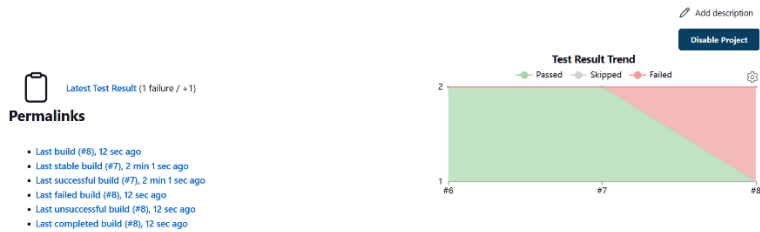
```
git commit -am "Simulating a stupid error !"
```

```
git push
```

Run the build again on this job in Jenkins. This time you should see the error reported in the Console Output for that particular build attempt.

The main area for the job should now indicate the build failure in the Test result trend (Refresh the page if it does not show) and also provide links to the various builds with various statuses (stable, successful, failed, unsuccessful, etc)

Project BasicGitHubDemo



The exact error details reported by the testing frame work is also visible from the Test Results view:

Return back to the app source code file:

```
src/main/java/com/mycompany/app/App.java
```

Restore the value of String MESSAGE to its original value: "I love Jenkins"
Save and exit.

As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Corrected my stupid error !"
```

```
git push
```

Run the build again. This time the build should be successful, and the various info on the main page for the job should reflect that as well (refresh the page if the Test result trend does not seem to change with the latest build result).

In the main area for the job, select Configure, add another Post-build action -> Archive the artifacts (note that this newly added action will appear right at the top of the list of the Post Build actions above the current existing one).

Provide the path of the files to archive relative to the main workspace folder: `target/*.jar`

We can also specify advanced options such as only archiving artifacts if the build is successful.

Post-build Actions

≡

Archive the artifacts ?

Files to archive ?

target/*.jar

Excludes ?

☐ Do not fail build if archiving returns nothing ?

☒ Archive artifacts only if build is successful

Click Save and run a build again.

If you now refresh the main page for the job, you should see a link for you to download the artifact.



Last Successful Artifacts

 [my-app-1.0-SNAPSHOT.jar](#) 2.67 KB [view](#)

This is very useful if the Jenkins server is running on a remote server, as it allows you to download the final build artifact to your local machine for further use.

11 Configuring periodic builds and SCM polling

At this point of time, we are manually performing our builds by clicking on the Build Now item in the main page for each job. This is not practical or ideal for real life projects, where team members may be pushing new changes to the central repo at variable times throughout the day.

Jenkins includes what is known as build triggers, which allows a build for a job to be triggered automatically when a specific event occurs. The most common situations would be:

- Periodically performing the build, for e.g. once every hour, or every 6 hours, or every day
- Checking the remote repo periodically for any updates (this is known as polling SCM) and then performing the build if there are any updates since the last poll. This is nearly the same as periodic builds, except that periodic builds will always occurs, regardless of whether there are any updates to the remote repo.
- When a specific event occurs (for e.g. a new commit is pushed to some branch on a remote repo, a pull request is opened to merge a feature branch back into the master branch, etc), the remote repo sends a signal (typically a HTTP POST request) to the Jenkins server to trigger the build. This feature is known as a webhook and is available with most Git cloud hosting services (GitHub, BitBucket, etc)

For the first 2 approaches, the scheduling is based on the [CRON job format from Linux](#). This is commonly used to [schedule the automated running of scripts](#) to perform system maintenance tasks periodically.

An example of the [CRON expression to run a job every minute](#)

The Jenkins CRON expression is [nearly similar to the Linux CRON syntax](#) with some minor differences:

In the main area for the previous job (Basic-GitHubJob), select Configure, go to Triggers, and select Build periodically. For the schedule, enter the CRON expression to run a job every minute:

* * * * *

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☒ Build periodically ?

Schedule ?

* * * * *

⚠ Do you really mean "every minute" when you say "* * * * *" ? Pe

Of course, in a real-life project, we would not run a build every 1 minute (this would consume too much resources on the build server !!), but for the purposes of demo in this workshop, we will set it to 1 minute. You can choose the CRON expression for the suitable interval of your choice in your real-life projects.

Click Save.

Go back to the main page for the job and wait for several minutes. Notice that a build will automatically be triggered every minute - **EVEN IF NO** updates have being made to the remote repo. You may have to refresh the main job page to see this periodical build being triggered every minute.

The build main page also explicitly indicates that the changes were started by the Jenkins timer (rather than manually by a user).

✓ **Build #10 (Feb 22, 2023, 11:16:00 PM)**

**Build Artifacts**

 my-app-1.0-SNAPSHOT.jar 2.54 KB [view](#)

 No changes.

 Started by timer

**Revision:** 4a8ef0400dc440a7539817d7a6a7221d55b2318
Repository: <https://github.com/victor-tan-hk/simple-java-app.git>

- refs/remotes/origin/master

 **Test Result** (no failures)

In the main area for the previous job (`Basic-GitHubJob`), select **Configure**, go to **Triggers**, and select **Poll SCM**. For the schedule, enter the CRON expression to run a job every minute, the same as previously.

Click **Save**.

Go back to the main page for the job and wait for 3 - 5 minutes. Notice that unlike before, no builds are triggered.

Return back to the original project location at: `simple-java-app`

Make a minor modification to the source code of the main class for this simple Java project at:

```
src/main/java/com/mycompany/app/App.java
```

Change the parameter of the `System.out.println` to some other random value
Save and exit.

As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Make a change to demonstrate Poll SCM"
```

```
git push
```

Go back to the main page for the job and wait for a minute. Now a build will be triggered because the polling of the GitHub repo indicates that there has been change in it since the last poll. Now leave it again for 2-3 minutes and notice that no further builds are triggered.

Notice as well that the build page clearly shows the build was triggered by a SCM change, and also provides details of the change (via the **Changes** link)

	✓ Changes Summary Make a change to demonstrate Poll SCM (details) Commit <code>c97b9dc5de80bf940c7e50a689194e387f600ee5</code> by <code>mainuser</code> Make a change to demonstrate Poll SCM src/main/java/com/mycompany/app/App.java (diff)
--	---

 Build #12 (Feb 22, 2023, 11:28:10 PM)  Build Artifacts  my-app-1.0-SNAPSHOT.jar 2.54 KB view  Changes 1. Make a change to demonstrate Poll SCM (details / githubweb)  Started by an SCM change  Revision: c97b9dc5de80bf940c7e50a689194e387f600ee5 Repository: https://github.com/victor-tan-hk/simple-java-app.git • refs/remotes/origin/master	
---	--

The main job page also shows the Changes detected in that Poll SCM.

Dashboard > user1-Basic-GitHubJob >

 Status  Changes  Workspace  Build Now	Changes #12 (Feb 22, 2023, 11:28:10 PM) 1. Make a change to demonstrate Poll SCM — mainuser / githubweb
---	--

Clicking on the GitHub web link links you directly to the commit details in the GitHub repo. All this additional info is provided back from a service in GitHub.

 Changes Summary 1. Make a change to demonstrate Poll SCM (details) Commit c97b9dc5de80bf940c7e50a689194e387f600ee5 by mainuser Make a change to demonstrate Poll SCM  src/main/java/com/mycompany/app/App.java (diff)	Dashboard > user1-Basic-GitHubJob > <table> <tr> <td>  Status  Changes  Workspace  Build Now </td> <td> Changes #12 (Feb 22, 2023, 11:28:10 PM) 1. Make a change to demonstrate Poll SCM — mainuser / githubweb </td> </tr> </table>	 Status  Changes  Workspace  Build Now	Changes #12 (Feb 22, 2023, 11:28:10 PM) 1. Make a change to demonstrate Poll SCM — mainuser / githubweb
 Status  Changes  Workspace  Build Now	Changes #12 (Feb 22, 2023, 11:28:10 PM) 1. Make a change to demonstrate Poll SCM — mainuser / githubweb		

When you are done verifying both these features, you can disable them in the Triggers section and Save the job again. We will enable them later if we need them in subsequent lab sessions.

12 Chaining jobs / projects

You can configure a build of Job B to execute immediately after the build of Job A completes. There are 2 common use cases for this:

- Job B has a dependency that is produced from the build for Job A (for e.g. a build artifact)
- Splitting a long build process into two (or more) smaller builds for easier management

Job B is considered a downstream project in this example, and we can perform the configuration in the Build Trigger section for Job B.

Terminology for upstream, downstream projects

From the main Jenkins dashboard, create a new Item and name it: `userx-GitHubJob-Followup`. Make this a Freestyle Project and select Ok.

In the Triggers Section, select Build after other projects are built, and enter `userx-Basic-GitHubJob`

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☒ Build after other projects are built ?

Projects to watch

user1-Basic-GitHubJob

☒ Trigger only if build is stable

In the Build Steps section, choose Add Build step -> Execute Shell commands, and add in the following Linux shell commands:

```
echo "Executing follow up job"
java -jar /var/lib/jenkins/workspace/userx-Basic-GitHubJob/target/my-app-1.0-SNAPSHOT.jar
```

Click Save, then return back to `userx-Basic-GitHubJob` and run a build on it. You should see a link to the chained job after the build completes on the main page.

Project user1-Basic-GitHubJob

 Last Successful Artifacts
my app 1.0 SNAPSHOT.jar 2.54 KB view

 Latest Test Result (no failures)

Downstream Projects

 user1-GitHubJob-Followup

Permalinks

If you check the Console Output for the latest build on `userx-Basic-GitHubJob`, you should see the triggering of `userx-GitHubJob-Followup` at the end.

```
Archiving artifacts
Recording test results
[Checks API] No suitable checks publisher found.
Triggering a new build of user1-GitHubJob-Followup
Finished: SUCCESS
```

However, you will not see any output from that chained / linked job. Instead, you must click on the link to transition to the main page of that job. On the main page of that job, you will see `userx-Basic-GitHubJob` marked as an upstream project.

Project user1-GitHubJob-Followup

Upstream Projects

user1-Basic-GitHubJob

Click on the particular build in the Build history for this chained job and you can see its Console Output shows that it is able to access the JAR in the workspace folder of the upstream job in order to execute it. This shows that jobs of a given user are able to access files / directories of other jobs created by the same user. You can create access controls to prevent such access if necessary (where multiple jobs are run by different users on a single Jenkins Server)

This chaining of jobs (Basic-GitHubJob -> GitHubJob-Followup) can also be accomplished alternatively by configuring Basic-GitHubJob instead of GitHubJob-Followup

In userx-GitHubJob-Followup, remove the Trigger that we created earlier and save.

Return to userx-Basic-GitHubJob and run another build. This time, userx-GitHubJob-Followup is not triggered.

In the configuration of this current job, in the Post Build Actions section, add another Post Build Action and select Build Other Projects and add userx-GitHubJob-Followup



Save and then run another build again on userx-Basic-GitHubJob

Again you should see the triggering of userx-GitHubJob-Followup in Console Output and you can link to it again in the usual way.

13 Configuring Webhooks with GitHub

In the previous labs, we looked at 3 different build triggers: Periodical builds, Poll SCM and Build after other projects are complete. Another popular and useful trigger is webhooks.

When a specific event occurs (for e.g. a new commit is pushed to some branch on a remote repo, a pull request is opened to merge a feature branch back into the master branch, etc), the remote repo sends a signal (typically a HTTP POST request) to the Jenkins server to trigger the build. This feature is known as a [webhook](#) and is available for [use with repos](#) on the main Git cloud hosting services (GitHub, BitBucket, etc)

At the Jenkins main dashboard, go to Manage Jenkins -> System -> Jenkins Location

The Jenkins URL is effectively the canonical public base URL for the controller, which is used for generating redirects when working with WebHooks. The Jenkins URL should already be set to the public IP address of the AWS EC2 instance running the Jenkin Server (check the address bar to verify this for sure).

Jenkins Location

Jenkins URL ?

http://54.179.215.54:8080/

Click Save when done.

To setup a WebHook for the GitHub repo `simple-java-app`,

Go the repo and Click Settings.

In the left sidebar, click Webhooks.

Click Add webhook.

For the Payload URL, enter:

<http://IP-address-JenkinsServer:8080/github-webhook/>

Make sure you type the webhook URL with an ending / , for e.g.

or else an error will occur when processing the HTTP POST request sent from GitHub on the Jenkins server end.

In the Content type select: `application/json`

Leave the Secret field empty.

Disable SSL verification as we have not set up HTTPS for our Jenkins server.

SSL verification

 By default, we verify SSL certificates when delivering payloads.

☐ Enable SSL verification ☒ Disable (not recommended)

For events to trigger the webhook, we stick to the default of just the push event.

Which events would you like to trigger this webhook?

- ☒ Just the push event.
- ☐ Send me **everything**.
- ☐ Let me select individual events.

For real life projects, you can click Let me select individual events, and decide which other events on the repo will trigger the webhook. Other common events besides a push are branch creation, commit comments, discussion comments, issues, pull requests and reviews and comments, etc.

Click Add Webhook.

Webhooks

Webhooks allow external services to be notified by a POST request to each of the URLs you provide.

- [http://43.217.114.95:8080/github-we... \(push\)](http://43.217.114.95:8080/github-webhook/)

This hook has never been triggered.

Click on the webhook entry above and select recent deliveries. This will show the results of GitHub sending a test ping to the Jenkins server at the specified URL and if you click on the ... menu at the end of the ping message, you should see the test request sent out and the successful response (HTTP code 200 OK) received in return.

✓
478edfc0-146a-11f0-8697-f9dc4c475916
ping
2025-04-08 19:11:56
...

Request	Response 200
Headers Request URL: <code>http://43.217.114.95:8080/github-webhook/</code> Request method: <code>POST</code> Accept: <code>*/*</code> Content-Type: <code>application/json</code> User-Agent: <code>GitHub-Hookshot/21a6bf0</code>	Headers Content-Length: <code>0</code> Date: <code>Tue, 08 Apr 2025 11:11:55 GMT</code> Server: <code>Jetty(12.0.18)</code> Vary: <code>Accept-Encoding</code> X-Content-Type-Options: <code>nosniff</code>

In the main page for the `userx-Basic-GitHubJob` select Configure, Triggers -> GitHub hook trigger for GitScm polling.

Click Save.

Wait for a few minutes. Notice that nothing happens.

Return to the project folder in `simple-java-app`

Let's make a minor modification to the source code of the main class at:

```
src/main/java/com/mycompany/app/App.java
```

Change the parameter of the `System.out.println` to some other random value, for e.g. `"Modified from local project !"`

Save and exit.

In the Git Bash shell in this project folder, check that the modifications have being made:

```
git status
```

Add the modifications to a new commit with an appropriate message:

```
git commit -am "Some changes to test webhooks"
```

Check the status again

```
git status
```

Return to the main job page for `userx-Basic-GitHubJob` in the Jenkins UI, and keep your eye on the Build History

Back in the Git Bash shell, push this latest commit to the master on the remote GitHub repo with:

```
git push
```

Notice now that a build is automatically triggered as a result of the webhook that you have setup. You may have to refresh the page to see this happen.

The status on the build main page clearly indicates the reason for the build being triggered (due to a GitHub push from a local repo by a user with the specified identity)

 **Build #16 (Feb 23, 2023, 8:14:25 AM)**



Build Artifacts
 `my-app-1.0-SNAPSHOT.jar` 2.55 KB [view](#)



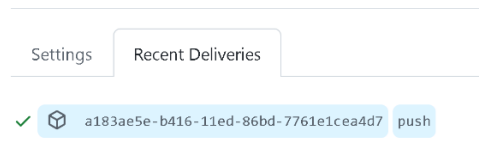
Changes
1. Some changes to test webhooks ([details](#) / [githubweb](#))



[Started by GitHub push by victor-tan-hk](#)

The GitHub Webhooks page for that particular WebHook should also correspondingly show the HTTP POST request that was sent out to the Jenkins server in order to trigger this build in the Recent Deliveries tab.

[Webhooks](#) / Manage webhook



Settings Recent Deliveries

  `a183ae5e-b416-11ed-86bd-7761e1cea4d7` [push](#)

You can drill down into the request and response by clicking on the ellipsis menu ... at the end of the entry to perform trouble shooting if necessary.

Webhooks / Manage webhook

The screenshot shows the 'Manage webhook' page for a repository. It has tabs for 'Settings' and 'Recent Deliveries'. Under 'Recent Deliveries', there is a green checkmark and a delivery ID 'a183ae5e-b416-11ed-86bd-7761e1cea4d7' for a 'push' event, dated '2023-02-24 15:41:22'. Below this, there are tabs for 'Request' and 'Response' (labeled '200'), and a 'Redeliver' button. A status bar indicates 'Completed in 0.76 seconds'. The 'Headers' section lists the following: Request URL: http://13.212.167.150:8080/github-webhook/, Request method: POST, Accept: */*, content-type: application/json, User-Agent: GitHub-Hookshot/3e3b786, X-GitHub-Delivery: a183ae5e-b416-11ed-86bd-7761e1cea4d7, X-GitHub-Event: push, X-GitHub-Hook-ID: 402284247, X-GitHub-Hook-Installation-Target-ID: 605076151, and X-GitHub-Hook-Installation-Target-Type: repository. The 'Payload' section is partially visible at the bottom.

14 Setting up a BitBucket repo

We will now create a new empty remote repo on BitBucket and push the contents of a local repo to populate it, the same way that we did for our GitHub repo.

Login to your Bitbucket account. You may have to create a new workspace if you are working with a new Bitbucket account.

Go to your Workspace Settings from the main workspace page, and make sure it is set to be Public.

The screenshot shows the Bitbucket 'Settings' page. On the left, under 'Atlassian Admin', there are links for 'User management' and 'Billing management'. Under 'Bitbucket Administration', 'Workspace settings' is highlighted. The main content area is titled 'Workspace settings' for 'victor33workspace'. It has two sections: 'General' and 'Privacy'. In the 'General' section, 'Workspace name' is 'victor33workspace' and 'Workspace ID' is 'victor33workspace'. In the 'Privacy' section, the 'Public' radio button is selected, indicating the workspace URL is visible to anyone.

Go through the process of [creating a new repository](#), but create a bare repository with no content to facilitate the process of pushing the contents of our local repository to it:

Enter the values for the following fields.

Project Name: MyJavaProject

Repository Name: simple-java-app

Make sure this is **NOT** a private repository, and **DON'T** include either a readme file or .gitignore. This is to ensure that the repository is a bare repository that we can push new content into from our local repo.

Create a new repository [Import repository](#)

Workspace Victor Tan

Project name* MyJavaProject

Repository name* simple-java-app

Access level ☐ Private repository
Uncheck to make this repository public. Public repositories typically contain open source code and can be viewed by anyone.

Include a README? No

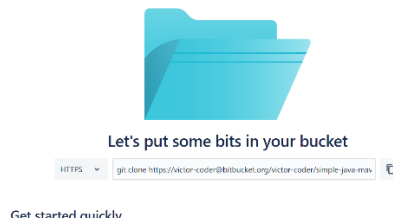
Default branch name e.g., 'main'

Include .gitignore? No

[> Advanced settings](#)

[Create repository](#) [Cancel](#)

When you are done specifying the values for the fields as shown above, click Create Repository. You will be transitioned to the Source view for the newly created repo, where some instructions will be provided on how to get started.



Click on the Copy button to copy the URL (e.g. `https://xxx@bitbucket.org/yyy/simple-java-maven-app.git`) for this new remote repo to an empty document. We will refer to this URL as *remote-url* in the commands to follow.

In your main `userx` folder, there is a `bitbucketprojects` folder, which in turn contains a `simple-java-app` folder. This is an exact copy of Java Maven project that we were working with earlier in the `githubprojects` folder

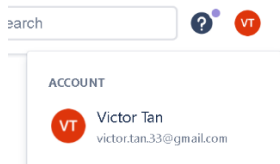
In the Bash shell, change to this folder with:

```
cd ~/userx/bitbucketprojects/simple-java-app
```

As we did earlier, initialize this as a new Git repo with:

```
git init
git add .
git status
```

Before creating the first commit in this newly initialized repository, you need provide an identify for the individual creating the commit (the commit author). Since we are going to be pushing our local repo to populate the BitBucket repo, we will use our BitBucket identity (name and email). This is accessible from the upper right hand corner of the page.



and note down the name and email address there. Use them in the configuration commands below:

```
git config --local user.name "your name"
```

```
git config --local user.email "your email address"
```

We are configuring these details at the local level so that they apply only to this repository, rather than to the user account since all users are now using this user account `mainuser`.

Double check that these configuration variable values have been set properly at the local level, which will be in the file `.git\config` in the current working directory

```
git config --local --list
```

Then we can create our first commit.

```
git commit -m "Added all project files in first commit"
```

```
git status
```

Next, we associate the remote repo URL with our newly initialized local repo with:

```
git remote add origin remote-url
```

Check that the origin handle is set to point to the correct repo URL with:

```
git remote --verbose
```

Before you can push the contents of the newly initialized repo to your remote BitBucket repo, you must first obtain [an API token](#). This is conceptually similar to classic personal access token (PAT) that we used in GitHub repo.

[Create the API token](#) and give it a suitable name and the scope type for the following Scope Actions: Write and Read.

Select Bitbucket scopes

Scopes allow you to choose the action an API token can perform in an app. You can select up to 50 scopes.

Search by scope name	Scope type ▾	Scope actions: Write ▾
☒	write:repository:bitbucket	Modify your repositories
☒	write:webhook:bitbucket	Modify your webhooks

Select Bitbucket scopes

Scopes allow you to choose the action an API token can perform in an app. You can select up to 50 scopes.

Scope type ▾
Scope actions: Read ▾

☒	read:pullrequest:bitbucket	View your pull requests
☒	read:repository:bitbucket	View your repositories
☒	read:webhook:bitbucket	View your webhooks

Click Next and verify that you have all the right scopes for Read and Write.

APPS

Bitbucket

SCOPES

Read
read:pullrequest:bitbucket
read:repository:bitbucket
read:webhook:bitbucket

Write
write:repository:bitbucket
write:webhook:bitbucket

Click Create Token.

When the API Token is shown in the dialog box, make sure you copy it down somewhere where you can access it before closing it.

Finally, push the entire contents of the local repo (including all its branches) to the remote repo with:

```
git push -u origin --all
```

You will be prompted for your app password:

```
Password for 'https://xxxxx@bitbucket.org':
```

You can copy and paste the API Token password, but be very careful to do this slowly and correctly.

Assuming you have entered the correct app password, the command should succeed with appropriate success messages in the Bash shell.

```
Enumerating objects: 39, done.
```

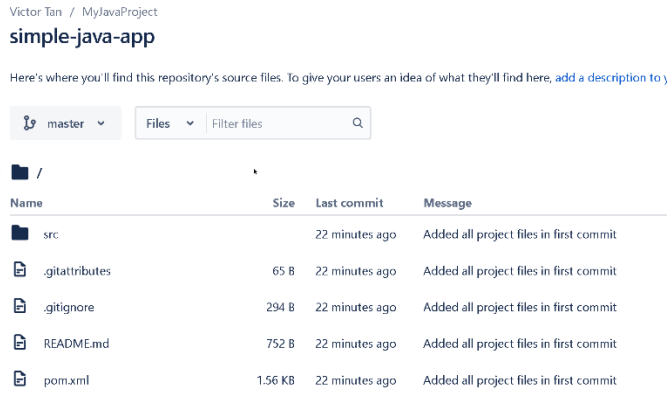
```
...
```

```
...
```

```
* [new branch]      master -> master
```

```
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Return back to the browser and check through the Source, Commits and Branches main view to verify that these mirrors the status of the local repo that you just pushed up into it. Note that you may need to navigate out to the main Repositories tab in the main menu, and then click on the `simple-java-app` repo entry in order for it to refresh properly and show the uploaded content. This may take some time to complete correctly.



Back in the Bash shell, type the following commands to verify that the remote tracking branches have been set up and that the local and upstream master are both in sync with each other.

```
git remote show origin
```

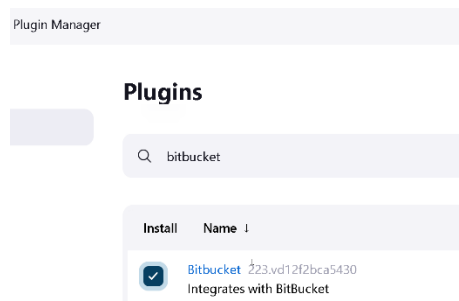
```
git status
```

15 Configuring Webhooks with BitBucket

We are now going to configure a WebHook for our BitBucket repo, which is conceptually the same as what we did for GitHub earlier.

Only one person with admin access needs to perform the plugin installation detailed below.

From the main Jenkins Dashboard, Manage Jenkins -> Plugins -> Available Plugins, look for BitBucket and select the item.



Select Install and click Restart Jenkins when installation is complete and no jobs are running once the download is complete. You may need to refresh the webpage to validate when the installation is complete, and login again.

As before, after the restart of the Jenkins server, go to the Installed Plugins section to verify that the Maven Integration Plugin has been included.

Name ↓	Enabled
Maven Integration plugin 3.25 This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit. Report an issue with this plugin	

From the main Jenkins dashboard, create a new Item and name it: `userx-Basic-BitbucketJob`. Make this a Freestyle Project and select Ok.

15.1 Exercise: Bitbucket WebHook configuration

As an exercise, set up the relevant steps / tasks in this new job to pull changes from your new BitBucket repo, run a Maven build to create a JAR artifact and then execute it. Finally, setup a configuration for a BitBucket Webhook and test it by pushing new commits from your local repo.

For new commits, make changes to

```
src/main/java/com/mycompany/app/App.java
```

in the correct `simple-java-app` folder, which is:

```
~/userx/bitbucketprojects/simple-java-app
```

Then add the modifications to a new commit with an appropriate message:

```
git commit -am "Suitable commit message of your own choosing"
```

Before we push the latest commit to the master on the remote BitBucket repo, we can optionally configure temporary caching of the username and API Token password at a local repository level for the same user account `mainuser` that everyone is working with. This eliminates the need to constantly have to reauthenticate for every push operation. The commands for this are:

```
git config credential.useHttpPath true
```

```
git config credential.helper "store --file .git/.credentials"
```

We can then push this latest commit to the master on the remote Bitbucket repo with:

```
git push
```

Paste in the API token, and this should be subsequently cached for later use.

Some sample guides to set up Bitbucket Webhook are shown below:

<https://support.atlassian.com/bitbucket-cloud/docs/manage-webhooks/>

<https://magefan.com/blog/add-webhook-in-bitbucket>

USEFUL TIPS:

- The WebHooks link is in the WorkFlow link in the Repository Settings

- You should ensure that the trigger `Build` when a change is pushed to Bitbucket is selected in the Triggers section of the configuration for your `userx-Basic-BitbucketJob`
- The webhook URL that you configure on BitBucket should be in this form:
`http://IP-address:8080/bitbucket-hook/`
- You should skip certificate verification in the Webhook configuration as the Jenkin service for our workshop is not secured with SSL/TLS
- In the Actions area, you can select View Requests

Repository hooks		
Title	URL	Actions
my cool webhook	http://56.68.38.193:8080/bitbucket...	View requests Edit Delete

- And then click on view request logs / Load new requests to see the history of requests send from BitBucket to the Jenkins service in response to the configured events (push commits to the remote repo). Unlike GitHub, Bitbucket does not perform an initial ping test request: in order to test out the webhook functionality, you will have to push a new commit into your remote Bitbucket repo.

16 Configuring email notification

Often, it is useful for the project lead / PIC in a software team to be updated every time a build fails. This is particularly important when the team is rushing to meet a product launch deadline or fixing a critical bug.

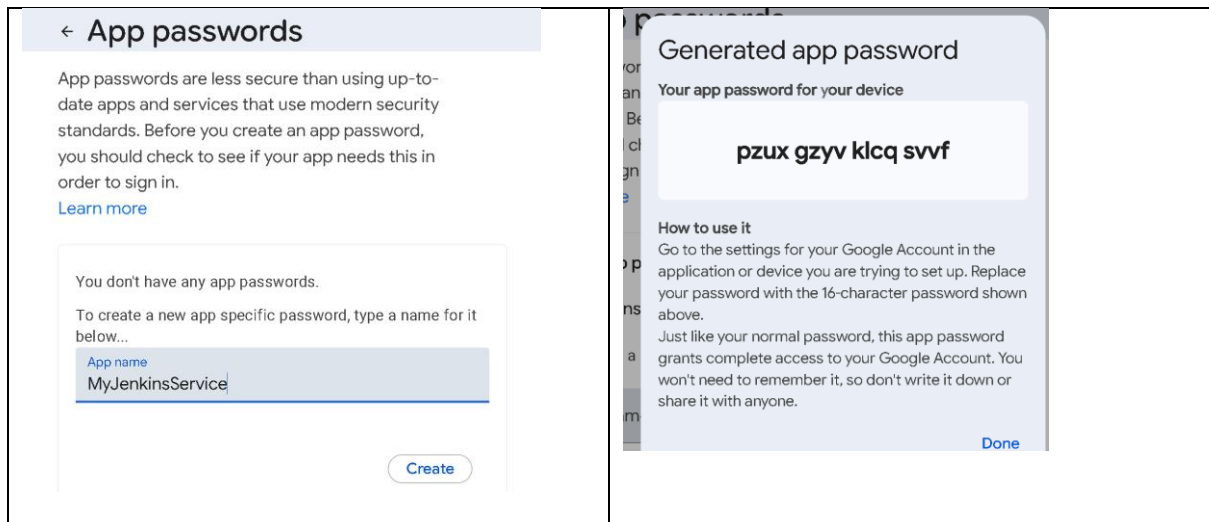
To configure email notification, Jenkins will need to use an SMTP server: this can be a free option available from public email hosting services like Gmail or internal to your company.

We will use the [free SMTP functionality offered with Gmail](#) as part of your Google Account.

The first step in configuring email notification on Jenkins is to [configure 2-Step Verification](#) for your Google Account (if you have not already configured this), and then [generate an App password](#) to be used in the configuration. You will need access to your Gmail password and linked phone number to perform these steps.

These are the screen shots of the App Password generation process:





Then return to the main Jenkins Dashboard, select Manage Jenkins -> Configure System and go to Jenkins Location and set an email address for the System admin. This email address will be used as the sender for all email notifications sent out from Jenkins.

Jenkins Location

Jenkins URL ?

http://3.1.81.98:8080/

System Admin e-mail address ?

mark.learnfast@gmail.com

Finally, scroll down to the Email Notification right at the bottom (NOT Extended Email Notification) and configure the details for the Email Notification for using the [Gmail SMTP settings](#). You can use a different email (or the same) as the System Admin that you set up earlier.

SMTP server: smtp.gmail.com

Username: your-gmail-address

Password: the app password you created earlier

Select Use SSL

SMTP port: 465

The most important point to note in this configuration process is that the password entered in the field is NOT the actual password associated with the email account specified in the User Name, rather it is the App Password that you generated earlier.

E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail suffix ?

☒ Use SMTP Authentication ?

User Name

mark.learnfast@gmail.com

Password

.....

☒ Use SSL ?☐ Use TLS

SMTP Port ?

465

Reply-To Address

mark.learnfast@gmail.com

Charset

UTF-8

☒ Test configuration by sending test e-mail

Test e-mail recipient

mark.learnfast@gmail.com

Perform a test configuration by sending test email. The test e-mail recipient can be any valid email address (and not just the username used in SMTP authentication).

If the test is successful, click Save.

In the main area for the previous job (userx-Basic-GitHubJob), select Configure, and add another Post-build action -> Email notification, enter a suitable email address (this would be the email for the person that is to be notified in the event of build failure) and check appropriate options for when emails will be sent and to whom.

E-mail Notification ?

Recipients

Whitespace-separated list of recipient addresses.

mail will be sent when a build fails, becomes unst

mark.learnfast@gmail.com

☒ Send e-mail for every unstable build

Click Save and manually perform a build.

The build is successful, so no notification email is sent.

Return back to the original project location at: `githubprojects/simple-java-app`

```
cd ~/userx/githubprojects/simple-java-app
```

Let's simulate an error occurring in the unit tests. In the file:

```
src/main/java/com/mycompany/app/App.java
```

Change the value of String MESSAGE to some other value, for e.g. "Dummy String"
Save and exit.

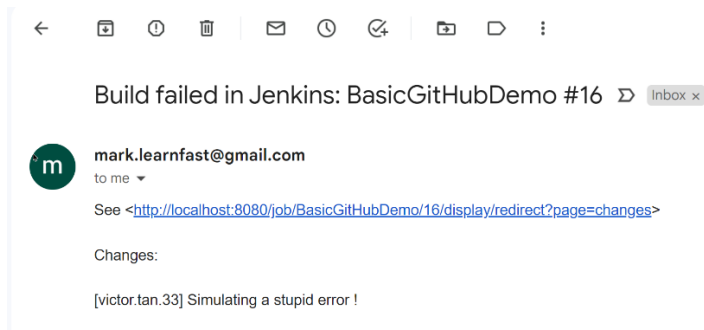
As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Simulating a stupid error !"
```

```
git push
```

Run the build again on Jenkins. The build will fail. If you check the Console Output, there will be a message sent to the registered user.

Check in that email account to verify that an email has being received from the email registered as the system admin in Jenkins Location, giving the details of the Console Output for that particular build attempt.



Let's correct the error we introduced deliberately earlier to prepare for future lab sessions. In the file:

```
src/main/java/com/mycompany/app/App.java
```

change the value of String MESSAGE back to the expected correct value: "I love Jenkins"
Save and exit.

As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Corrected the stupid error !"
```

```
git push
```

Run the build again on Jenkins. This time the build succeeds, and no notification message is sent out.

In addition to Gmail, there are many other organizations that provide free SMTP servers to cater for this kind of situation.

<https://moosend.com/blog/free-smtp-server/>

<https://www.emailvendorselection.com/free-smtp-servers/>

Your company / organization may also have its own internal SMTP server that you can make use of:

In addition to notifications via email, Jenkins provides plugins that support integration with other project management / collaboration tools for the purposes of build notifications (for e.g. [Slack](#) and [Jira](#))